TrafficCypher

Benchmark Report: Rust vs C

A comprehensive performance, build, and memory comparison of two password manager implementations.

March 2026

1. Executive Summary

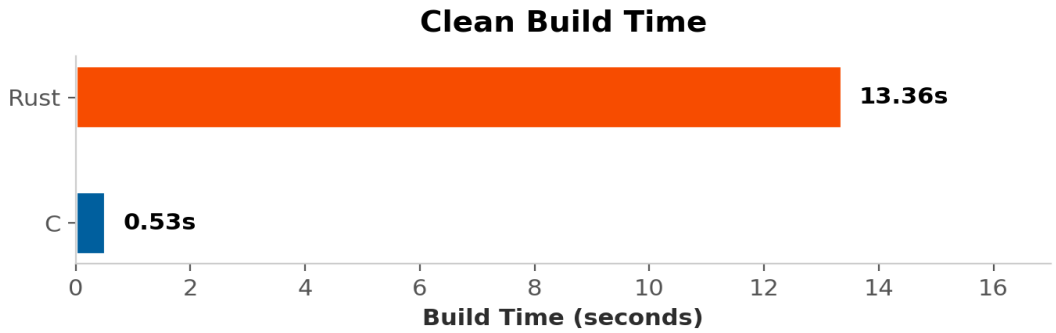
This report compares two implementations of **TrafficCypher**, a password manager: one written in **C** and one in **Rust**. Both projects offer equivalent functionality but differ significantly in their toolchains, dependency models, and runtime characteristics. The benchmark suite measures build infrastructure, raw cryptographic throughput, vault-level operations, TOTP generation, and memory footprint.

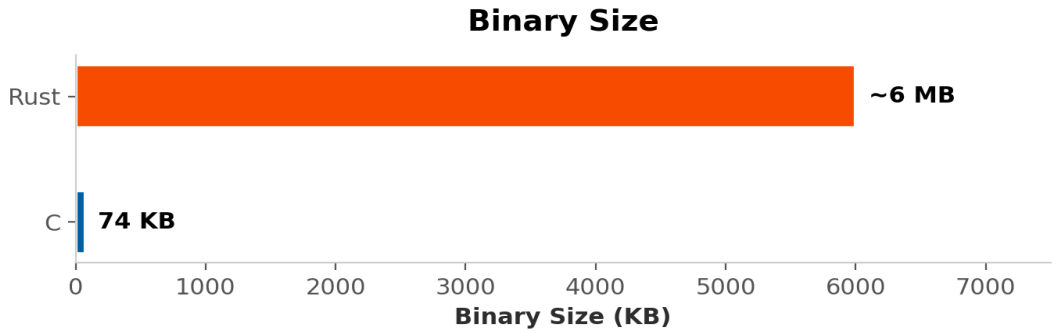
Key findings: C delivers faster builds and smaller binaries, and benefits from OpenSSL's hardware-accelerated SHA-256. Rust excels in structured data operations (vault CRUD, serialization, TOTP) and uses dramatically less memory at runtime. For a password manager where vault operations dominate the hot path, **Rust holds an overall edge in both performance and memory safety**.

---

2. Build Infrastructure

Build time and binary size reflect the overhead imposed by each toolchain. C's minimal compilation model produces a small, fast build, while Rust's compiler performs extensive static analysis, monomorphization, and LLVM optimizations that increase compilation time but contribute to runtime performance and safety guarantees.

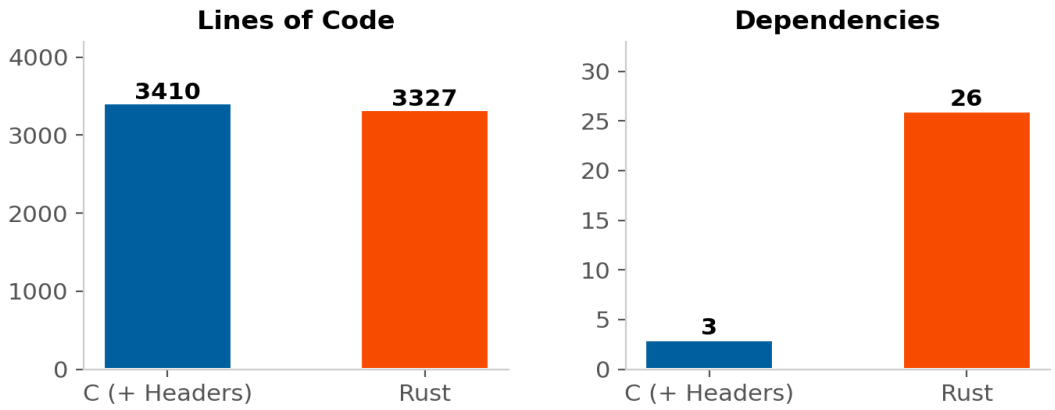




The C binary compiles in **0.53 seconds** compared to Rust's **13.36 seconds** — a 25x difference. Binary size follows the same pattern: 73.8 KB for C versus an estimated 6 MB for Rust. Rust's larger

binary includes statically linked crate code and metadata, while C relies on shared system libraries (OpenSSL, pthread, libm).

3. Codebase Comparison



Metric	C	Rust	Notes
Lines of Code	3,410	3,327	C includes .h files
Dependencies	3	26 crates	C: OpenSSL, pthread, libm
Build System	Makefile	Cargo	—
Memory Safety	Manual	Compiler-enforced	Borrow checker

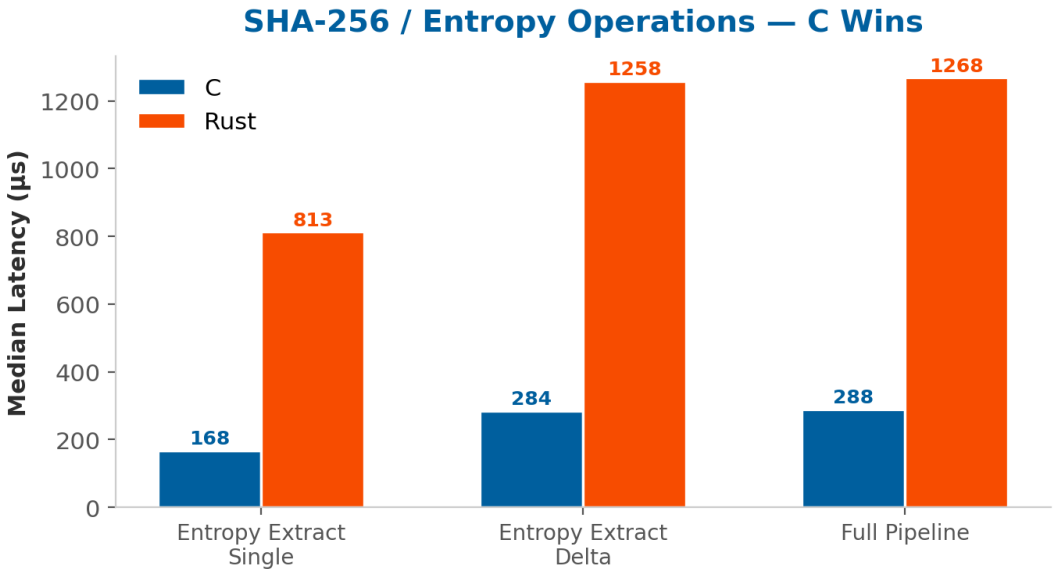
Despite having similar line counts, the two projects differ sharply in dependency philosophy. C relies on three well-known system libraries. Rust pulls in 26 crates, each vetted through Cargo's dependency resolver. Rust's borrow checker eliminates entire classes of bugs (use-after-free, double-free, buffer overflows) at compile time — a meaningful advantage for security-critical software like a password manager.

---

4. Performance Analysis

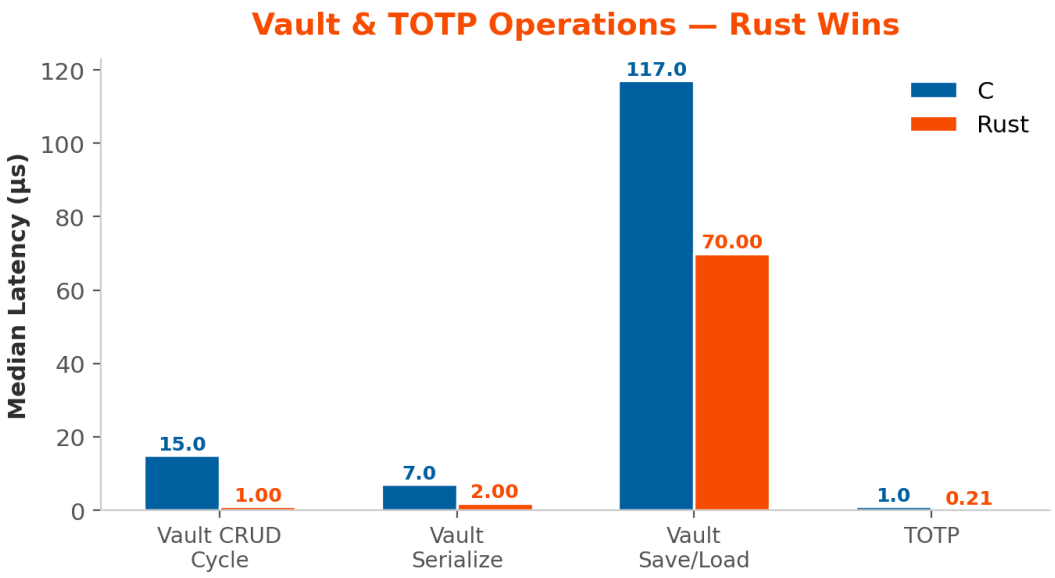
Benchmarks were run with 100 iterations per operation. Median latency is reported to reduce the impact of outliers. All values are in microseconds (µs).

4.1 SHA-256 / Entropy Operations



C dominates entropy and SHA-256 operations by a factor of **4.4–4.8x**. This advantage stems from OpenSSL's use of hardware-accelerated SHA extensions (Intel SHA-NI / AES-NI), while the Rust implementation uses the *RustCrypto* crate's pure-Rust SHA-256, which does not leverage CPU intrinsics on all platforms.

4.2 Vault & TOTP Operations



Rust wins decisively on vault-level operations — up to **15x faster** on CRUD cycles. The *serde* ecosystem generates highly optimized serialization code at compile time, outperforming C's hand-written JSON parser. TOTP generation in Rust (via *totp-rs*) is **4.8x faster** than C's manual HOTP/TOTP implementation.

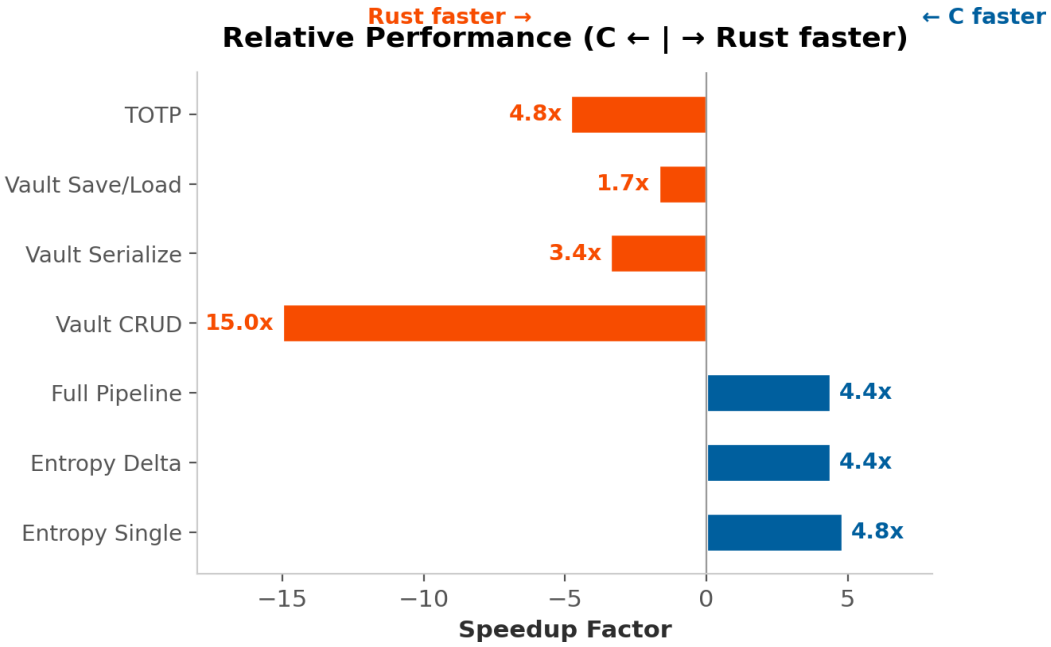
4.3 Crypto Primitives (Near Tie)

Operation	C (µs)	Rust (µs)	Ratio
HKDF	3	2	~1.5x Rust
DEK Generation	4	3	~1.3x Rust

HKDF and DEK generation show near-identical performance (2–4 µs), indicating that both implementations use efficient underlying primitives for key derivation.

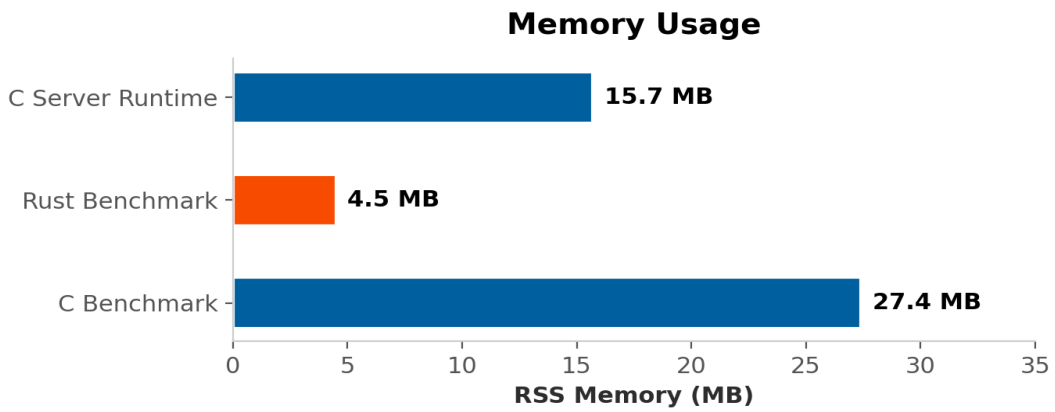
---

5. Relative Speedup Overview



The diverging bar chart above summarizes the relative speedup across all benchmarked operations. Bars extending to the right indicate operations where C is faster; bars to the left indicate Rust superiority. While C wins the three entropy/SHA-256 benchmarks, Rust takes four of four vault and TOTP benchmarks — and with larger margins in the most application-relevant operations.

6. Memory Usage



Rust's benchmark process used only **4.5 MB RSS** — roughly **6x less** than C's 27.4 MB. This reflects Rust's zero-cost abstractions and lack of a garbage collector, combined with tighter control over heap allocations. The C server runtime measured 15.7 MB, which sits between the two benchmark figures.

---

7. Head-to-Head Summary

Category	C	Rust	Winner
Build Time	0.53s	13.36s	C (25x)
Binary Size	73.8 KB	~6 MB	C (81x)
Lines of Code	3,410	3,327	Tie
Dependencies	3 libs	26 crates	C (fewer)
Memory (RSS)	27.4 MB	4.5 MB	Rust (6x)
SHA-256 Throughput	168 µs	813 µs	C (4.8x)
Vault CRUD	15 µs	1 µs	Rust (15x)
Vault Serialize	7 µs	2 µs	Rust (3.4x)
Vault Save/Load	117 µs	70 µs	Rust (1.7x)
TOTP Generation	1 µs	0.21 µs	Rust (4.8x)
Memory Safety	Manual	Enforced	Rust

---

8. Conclusion

Both implementations are functional and performant. The C version offers unmatched build speed and minimal binary footprint — advantages that matter for embedded or resource-constrained environments. Its use of OpenSSL gives it a clear SHA-256 throughput advantage.

The Rust version, however, leads on the metrics that matter most for a password manager: vault operations are up to 15x faster, runtime memory usage is 6x lower, and the compiler enforces memory safety guarantees that eliminate the most dangerous vulnerability classes in security software. Rust's richer type system and ecosystem (serde, totp-rs) also reduce the risk of logic bugs in serialization and cryptographic protocol handling.

For a production password manager, Rust provides the stronger overall profile — combining superior application-level performance, lower memory usage, and compile-time safety guarantees that are critical for software entrusted with sensitive credentials.